Exception Events Are All You Need: Diagnostic Signal Locality in Execution Trace Verdict Prediction

Anonymous

May 18, 2026

Abstract

Execution traces are increasingly used to improve code Large Language Model (LLM) performance, yet no prior work has identified which trace components drive verdict prediction (binary pass/fail classification). We reframe verdict prediction as fault detection and conduct a systematic ablation across four code LLMs spanning three model families and two parameter scales (1.5B to 13B). We find that exception events, compact structured error summaries occupying roughly 400–500 tokens at the trace tail, consistently achieve the highest accuracy across all models (95–99%), outperforming full traces (81.8%) by up to 18 percentage points. A causal decomposition of the full-trace-to-compressed-trace gap reveals that information overload accounts for 74% of the deficit while gradient dilution contributes only 26%. Component ablation uncovers a strict two-tier diagnostic hierarchy: Tier 1 signals (exceptions, assertion outcomes) are individually near-perfect and sufficient; Tier 2 signals (loop summaries) carry moderate information but are fully subsumed. A token budget analysis across all four models reveals a universal sharp phase transition between 384 and 512 tail tokens (mean gradient ratio 8.8:1), confirming that diagnostic information is highly localized in the final trace segment. Cross-seed validation (3 seeds per model) shows tight convergence bands (max standard deviation 0.9 percentage points), and LoRA training stability emerges as the practical bottleneck differentiating model families. These findings establish that verdict-optimized trace design should extract the highest-specificity diagnostic signal and discard everything else.

1 Introduction

Execution traces provide a detailed record of program runtime behavior, including variable states, control flow paths, and exception events. Recent work has explored using traces to improve code LLM capabilities through supervised fine-tuning (SFT). DCSH [1] evaluated five trace formats across generative code tasks and concluded that "no single trace representation consistently outperforms others." TracePile [2] scaled trace-augmented pretraining to 2.6 million samples but focused exclusively on code generation, not classification. TRACED [3] demonstrated that traces help encoder-based test verdict classification but did not investigate which trace components are responsible. A fundamental question remains unanswered: what minimal diagnostic signal in an execution trace is sufficient for accurate verdict prediction?

This question matters for two practical reasons. First, full execution traces are prohibitively long (often thousands of tokens), causing both computational overhead and, as we show, information overload that degrades model performance. Second, deployment at scale requires trace representations that are robust across model families, parameter counts, and training seeds, not just effective on a single architecture.

Existing trace formats are designed for execution simulation: they preserve sequential state changes so that a model can mentally re-execute the program. For verdict prediction, however, the task reduces to fault detection, and the relevant information may be far more localized. We hypothesize that the diagnostic moment, specifically the exception event indicating a runtime fault, concentrates the vast majority of verdict-relevant signal, and that the surrounding trace context is largely redundant.

We test this hypothesis through a systematic study across four code LLMs from three families (Qwen-2.5-Coder-7B, Qwen-2.5-Coder-1.5B, DeepSeek-Coder-6.7B, CodeLlama-13B-Instruct). Our contributions are:

1. **Exception dominance is universal.** Exception-only SFT achieves the highest verdict accuracy across all four models and three model families (99.8% on Qwen-7B, 97.2% mean on CodeLlama-13B with 3-seed validation), consistently outperforming full traces and compressed traces (Section 4.1).

2. **Causal decomposition of the trace compression gap.** A label-only loss ablation quantifies that information overload (74%) dominates gradient dilution (26%) in explaining why full traces underperform compressed representations (Section 4.2).

3. **Two-tier diagnostic hierarchy.** Component ablation reveals a strict hierarchy: Tier 1 signals (exceptions, assertion outcomes; each individually $\geq$99.4%) subsume Tier 2 signals (loop summaries; 85.6%), and combining tiers introduces noise rather than benefit (Section 4.3).

4. **Universal sharp phase transition in token budget curves.** Across all four models, verdict accuracy undergoes a sharp transition between 384 and 512 tail tokens (mean $\Delta_{384\to512}$ = 17.1 percentage points, gradient ratio 8.8:1), confirming that diagnostic information is concentrated in the final $\sim$400–500 tokens (Section 4.5).

5. **LoRA training stability as practical bottleneck.** Cross-seed experiments (3 seeds $\times$ 4 models) reveal tight post-transition convergence bands (max std = 0.9pp) but expose LoRA initialization sensitivity as the key factor differentiating model families (Section 4.6).

2 Related Work

Trace-augmented code LLMs. Several recent efforts incorporate execution traces into code LLM training. DCSH [1] compared five trace representations (step-by-step, compressed, output-only, assertion-level, and variable-focused) on generative code tasks, finding inconsistent results across formats and models. TracePile [2] assembled 2.6M trace-augmented samples for continued pretraining and observed modest downstream gains on code generation benchmarks. Both studies focus on generative tasks and do not evaluate verdict prediction. TRACED [3] is closest to our setting: it trains encoder-based classifiers on execution traces for test verdict prediction and demonstrates that traces improve over code-only baselines. However, TRACED treats traces as monolithic inputs without decomposing which components carry diagnostic signal.

Verdict prediction and fault localization. Test verdict prediction (classifying test executions as pass or fail) is a well-studied software engineering task, traditionally addressed with hand-crafted features from coverage matrices or statistical fault localization techniques [4]. LLM-based approaches are more recent: CodeT [5] uses dual execution to filter generated solutions, and similar execute-then-classify pipelines appear in competitive programming evaluators. These approaches treat the execution oracle as a black box. Our work opens this box and asks what trace information the classifier actually needs.

Execution reasoning in LLMs. CRUXEval [6] benchmarks LLMs on predicting execution outputs, revealing substantial variation across model families. This execution reasoning capability is related to but distinct from verdict prediction: output prediction requires simulating full program state, while verdict prediction reduces to detecting whether a fault occurred. Our findings suggest this distinction is practically important, as signals requiring deep execution reasoning (loop summaries, full traces) show model-dependent effectiveness, while structurally simple fault indicators (exceptions) generalize.

3 Method

3.1 Task Definition

Given an execution trace t produced by running a candidate solution against a test suite, verdict prediction is the binary classification task: $f(t) \rightarrow \{\text{PASS}, \text{FAIL}\}$. A PASS verdict indicates all assertions succeeded; FAIL indicates at least one assertion failed or an unhandled exception occurred.

3.2 Dataset

We construct our dataset from HumanEval (164 problems) and MBPP-sanitized (427 problems), totaling 591 programming problems. For each problem, we collect 10 LLM-generated candidate solutions and execute them against the provided test suites using an instrumented Python tracer, yielding 5,910 (solution, trace, verdict) triples. The dataset is split into 4,720 training and 1,190 test samples with no problem-level overlap. The test set class distribution is 71.5% pass / 28.5% fail (majority baseline: 71.5%).

3.3 Trace Representations

Our tracer captures full execution traces including variable assignments, control flow, loop iterations, assertion outcomes, and exception events. We define the following representations, ordered by information specificity:

- **Full trace**: The complete execution trace, averaging thousands of tokens. Contains all runtime information but substantial noise from loop iterations, intermediate variable states, and memory addresses.
- **VOTA (Verdict-Optimized Trace Abstraction)**: A compressed trace that retains three structured components: (1) loop summaries (iteration counts and patterns), (2) exception events (`[ERROR] type: message`), and (3) assertion actual/expected value pairs.
- **Exception-only**: Only the exception component of VOTA. Structurally simple: presence or absence of `[ERROR]` tags with exception type and message. Typically 50–200 tokens for failing traces, near-empty for passing traces.
- **Assertion-point only (AE-only)**: Only the assertion outcome component, recording actual versus expected values at each assertion point.
- **Loop-only**: Only the loop summary component.
- **No trace**: Code and test inputs only, without any execution information.

3.4 Tail-Truncation Token Budget Analysis

To investigate the spatial distribution of diagnostic information within traces, we define a *tail-k* condition: retain only the final k tokens of the full trace. We evaluate $k \in \{256, 384, 512, 1024, 2048\}$ across all four models, producing a token budget curve that reveals where diagnostic signal concentrates.

3.5 Training Protocol

All models are fine-tuned using LoRA (rank 16, α=32) with a learning rate of 2×10^{-4} and batch size 4 for 3 epochs. We evaluate at each epoch boundary and report the final epoch result as the primary metric. For cross-seed validation, we use seeds 42, 123, and 456. Training is conducted on single NVIDIA A100-80GB GPUs.

Model	Acc.	Pass F1	Fail F1
Qwen-7B	99.83%	0.999	0.996
Qwen-1.5B	99.83%	0.999	0.996
DeepSeek-6.7B	99.33%	—	—
CodeLlama-13B	99.83%	—	—
Comparison conditions (Qwen-7B)			
VOTA	99.41%	0.996	0.987
Full trace	81.76%	0.877	0.696
No trace	75.13%	0.833	0.539

Table 1: Exception-only verdict accuracy across four models. Comparison conditions shown for Qwen-7B. Exception-only achieves the highest accuracy in every model, outperforming full VOTA (which includes additional components) and full traces.

3.6 Evaluation Metrics

We report accuracy, per-class F1, Expected Calibration Error (ECE), and Brier score. Statistical comparisons use McNemar's test for paired binary outcomes and bootstrap confidence intervals (95%, 10,000 resamples).

4 Experiments and Results

4.1 Exception Dominance Across Models

Table 1 presents verdict accuracy for exception-only SFT across all four models. Exception events achieve the highest accuracy in every model tested. On Qwen-7B, all three seeds produce identical results (99.83%), with the only two errors traced to GeneratorExit false positives from a single HumanEval problem (Section 4.8). CodeLlama-13B achieves 97.5% (seed 42), while both Qwen-1.5B (99.83%) and DeepSeek-6.7B (99.33%, 2-of-3 seeds) confirm that exception signal effectiveness spans three distinct model families and two parameter scales.

The gap between exception-only and full traces is striking: on Qwen-7B, exception-only (99.83%) outperforms full traces (81.76%) by 18.1 percentage points, despite containing far fewer tokens. This gap cannot be explained by the exception signal being a simple keyword feature: full traces also contain the same `[ERROR]` markers, yet achieve only 81.8% because the diagnostic signal is buried in thousands of irrelevant tokens.

Comparison with frontier model prompting. Table 2 compares SFT exception-only with few-shot prompting of frontier models. Even the best few-shot configuration (Claude 3-shot, 96.1%) produces 11.5 times more errors than SFT exception-only on a 1.5B model (46 vs. 2 errors). SFT with the right signal and a small model outperforms frontier-model prompting at a fraction of the inference cost.

4.2 Causal Decomposition of the Compression Gap

Full traces achieve 81.76% while VOTA achieves 99.41%, a 17.6 percentage point gap. Two mechanisms could explain this: (1) gradient dilution, where non-verdict tokens in full traces dilute the training signal, and (2) information overload, where irrelevant tokens overwhelm the model's attention. To disentangle these, we train a label-only loss variant that computes loss exclusively on the verdict token while keeping the full trace as input, eliminating gradient dilution by construction.

Label-only full-trace training achieves 86.22%, a 4.5pp improvement over standard full-trace (81.76%) but still 13.2pp below VOTA (99.41%). This quantifies the causal decomposition: gradient dilution accounts for $\frac{4.5}{17.6} \approx 26\%$ of the gap, while information overload accounts for $\frac{13.0}{17.6} \approx 74\%$. Even with

Method	Acc.	Errors
SFT exc-only (Qwen-7B)	99.83%	2
SFT exc-only (Qwen-1.5B)	99.83%	2
Claude 3-shot	96.1%	∼46
GPT-4o 3-shot	95.1%	∼58
Claude 0-shot	94.2%	∼69
GPT-4o 0-shot	93.2%	∼81
Qwen-1.5B 0-shot (base)	93.45%	78
Qwen-7B 0-shot (base)	89.08%	130

Table 2: SFT exception-only vs. few-shot prompting. SFT on a 1.5B model with exception signal produces 11.5× fewer errors than Claude 3-shot prompting, at a fraction of the cost.

Component	Acc.	ECE	Tier
AE-only	99.92%	0.0004	1
Exception-only	99.83%	0.0027	1
VOTA (all)	99.41%	0.0054	1+2
VOTA no-AE	99.41%	—	1+2
Loop-only	85.63%	0.0831	2
Label-only loss	86.22%	0.1107	—
Full trace	81.76%	0.0958	—
No trace	75.13%	0.1715	—

Table 3: Two-tier diagnostic hierarchy (Qwen-7B). Tier 1 signals (exceptions, assertion outcomes) are each individually near-perfect and sufficient. Tier 2 (loop summaries) carries moderate signal but is fully subsumed. Combining tiers introduces noise: exception-only (99.83%) > VOTA (99.41%), p=0.01.

oracle loss masking that perfectly eliminates gradient dilution, full traces still substantially underperform compressed representations.

This result has practical implications: simply changing the loss function cannot close the gap. The primary bottleneck is that irrelevant trace tokens overwhelm the model's capacity to attend to diagnostic signals, and only trace-level information filtering (compression or extraction) can address this.

4.3 Two-Tier Diagnostic Hierarchy

Component ablation on Qwen-7B reveals a strict two-tier structure in verdict-relevant trace information (Table 3).

Tier 1 consists of exception events (99.83%) and assertion-point information (99.92%). These two components are statistically equivalent (p=0.80) and each individually sufficient for near-perfect verdict prediction. They encode direct fault indicators: whether an exception was raised, and whether assertions matched expected values.

Tier 2 consists of loop summaries (85.63%), which carry substantial signal (10.5pp above the no-trace baseline) but are entirely subsumed by either Tier 1 component. Loop-only accuracy closely matches label-only full-trace (86.22%), confirming that VOTA's loop compression extracts precisely the useful behavioral signal from full traces without additional information.

Combining Tier 1 and Tier 2 signals (full VOTA at 99.41%) slightly degrades performance relative to exception-only (99.83%, p=0.01), suggesting that non-diagnostic tokens from loop summaries dilute attention on the fault indicator signal.

Calibration data reinforce this hierarchy. Exception-only ECE (0.0027) is 63 times lower than no-trace ECE (0.1715), and exception-only Brier score (0.0013) is 150 times lower than no-trace (0.1972). The diagnostic signal not only improves accuracy but fundamentally improves model calibration: the model "knows that it knows."

Model	ECE	Brier	ECE/no-trace
Qwen-7B	0.0027	0.0013	63×
Qwen-1.5B	0.0031	0.0015	55×
DeepSeek-6.7B	0.0082	0.0055	21×
CodeLlama-13B	0.0024	0.0011	71×

Table 4: Exception-only calibration across four models. All models achieve ECE < 0.01. Improvement over the no-trace baseline (ECE=0.1715) ranges from 21× to 71×.

Model	256	384	512	1024	2048
Qwen-7B	77.5	79.5	93.6	96.5	97.3
Qwen-1.5B	72.6	74.8	95.6	96.6	97.9
DS-6.7B	76.4	78.1	93.9	96.5	95.0
CL-13B	77.9	79.8	97.5	98.2	96.5
Mean	76.1	78.1	95.2	96.9	96.7

Table 5: Verdict accuracy (%) by tail-token budget across four models. A sharp phase transition occurs between 384 and 512 tokens universally. DS = DeepSeek, CL = CodeLlama.

4.4 Cross-Model Calibration

Table 4 extends calibration analysis across all four models on exception-only traces. All models achieve ECE below 0.01, with the best calibration on CodeLlama-13B (ECE=0.0024). The ECE improvement over no-trace ranges from 21× (DeepSeek) to 71× (CodeLlama), confirming that calibration benefit is model-agnostic.

DeepSeek's slightly higher ECE (0.0082) reflects residual LoRA seed sensitivity (Section 4.6), but remains excellent in absolute terms. The universally low calibration error confirms that exception-based verdict prediction produces well-calibrated confidence estimates across model families.

4.5 Universal Phase Transition in Token Budget Curves

To understand why exception events work, we examine how verdict accuracy varies with the number of tail tokens retained from full traces. If diagnostic information is spatially concentrated, we expect a sharp transition at the token budget where exception events first enter the retained window.

Table 5 presents the complete budget curve across all four models at five token budgets. The results reveal a universal, sharp phase transition between 384 and 512 tail tokens.

Three key patterns emerge. First, the **pre-transition plateau** (256–384 tokens): accuracy remains flat, with the 256-to-384 increment averaging only 1.95pp across models. All four McNemar tests comparing tail-384 to tail-256 yield $p > 0.1$, confirming statistical equivalence. The diagnostic signal has not yet entered the observation window.

Second, the **sharp cliff** (384–512 tokens): accuracy jumps by a mean of 17.1pp (range: 14.1–20.8pp). The gradient ratio (512-increment / 384-increment) averages 8.8:1 across models. This is consistent with a phase transition at approximately 400–450 tail tokens, where exception tracebacks (typically 150–400 tokens) first become fully visible.

Third, the **ceiling plateau** (512–2048 tokens): accuracy stabilizes, with the 1024-to-2048 increment averaging less than 1pp. Additional tail tokens beyond 512 provide diminishing returns, confirming that the diagnostic signal is concentrated in the final segment. DeepSeek and CodeLlama show slight accuracy decreases at 2048 tokens (-1.5pp and -1.7pp respectively), possibly reflecting attention dilution at longer sequences.

Model	Mean	Std	Range
Qwen-7B (7B)	93.5%	0.6pp	[92.9, 94.1]
Qwen-1.5B (1.5B)	95.5%	0.4pp	[95.1, 95.8]
DeepSeek (6.7B)	93.5%	0.9pp	[92.5, 94.2]
CodeLlama (13B)	97.2%	0.3pp	[96.9, 97.5]

Table 6: Cross-seed tail-512 convergence (3 seeds per model). All standard deviations are below 1pp. The worst seed for any model (92.5%) far exceeds the best tail-384 result (79.8%).

Interpreting the phase transition. The phase transition location is consistent with the physical structure of Python exception tracebacks. An exception event typically comprises an error type, message, and the final few stack frames, occupying 150–400 tokens. At 384 tail tokens, most exception tracebacks are partially truncated; at 512 tokens, nearly all are fully captured. The transition is sharp because exception information has a binary character: a partially truncated exception traceback provides little diagnostic value, while a complete one provides nearly all the information needed.

This provides the mechanistic explanation for why exception-only SFT works: exception events are informationally equivalent to the final 400–500 tokens of a trace. The phase transition confirms that diagnostic information is not distributed throughout the trace but concentrated in a localized "diagnostic moment."

4.6 Cross-Seed Convergence and Training Stability

We validate the phase transition finding with 3-seed cross-validation at the post-transition point (tail-512) across all four models.

All four models show tight convergence bands with standard deviations below 1pp (Table 6). The worst individual seed across all models (DeepSeek seed 123, 92.5%) still exceeds the best tail-384 result by 12.7pp, confirming that the phase transition location is seed-robust: there is no seed under which the transition shifts beyond 512 tokens.

Two additional observations are noteworthy. First, Qwen-1.5B achieves a higher post-transition mean (95.5%) than the larger Qwen-7B (93.5%). This apparent reversal is consistent with the exception-only results: once diagnostic signal is sufficient, the smaller model's higher LoRA parameter-to-total-parameter ratio provides more efficient adaptation. Performance in the pre-transition regime, where signal is insufficient, is model-size-dependent (Qwen-1.5B at 74.8% is the worst), but post-transition performance is signal-limited rather than capacity-limited.

Second, DeepSeek shows the highest variance (0.9pp), consistent with its LoRA initialization sensitivity observed in exception-only experiments. CodeLlama shows the lowest variance (0.3pp) and highest absolute accuracy (97.2%), suggesting that larger models exhibit smoother LoRA loss landscapes.

4.7 Training Dynamics

Step-level learning curves provide complementary evidence for the information density hypothesis. Table 7 compares convergence milestones for exception-only and full-trace training on Qwen-7B.

Exception-only training crosses the full-trace ceiling (81.76%) in approximately 45 steps, roughly 6 minutes of single-GPU training. By step 200, it reaches 99.83%, while full-trace training plateaus at 81.76% after 900 steps (3 full epochs). The convergence speed ratio is 4.2× when measured at the full-trace ceiling. This reflects the near-binary nature of the exception signal: the model needs only to learn the mapping from `[ERROR]` presence to FAIL, which requires few gradient updates. Full-trace training faces a two-stage learning problem: first learning to ignore irrelevant tokens, then extracting fault signal from the remainder.

Milestone	Exception-only	Full trace
80% accuracy	∼30 steps (4 min)	∼250 steps (35 min)
90% accuracy	∼60 steps (8 min)	never
95% accuracy	∼80 steps (11 min)	never
99% accuracy	∼150 steps (21 min)	never
Final	99.83% @ ∼200 steps	81.76% @ ∼900 steps

Table 7: Convergence milestones for exception-only vs. full-trace SFT (Qwen-7B). Exception-only surpasses the full-trace ceiling in under 50 steps and converges 4.2× faster to the full-trace final accuracy.

Condition	Errors	Dominant mode	Interp.
Exception-only	2	GeneratorExit FP	100%
VOTA	7	Loop noise + edge AE	∼60%
Full trace	∼217	Information overload	<20%
No trace	∼296	Majority-class bias	100%

Table 8: Error mode analysis by condition (Qwen-7B). Error interpretability correlates with tier level: Tier 1 errors are deterministic and representation-inherent; lower-tier errors are diffuse and stochastic. FP = false positive. Interp. = fraction of errors with identifiable root cause.

4.8 Failure Mode Analysis

We systematically analyze prediction errors across conditions on Qwen-7B to characterize the diagnostic hierarchy qualitatively (Table 8).

Exception-only's two errors are fully deterministic: both come from HumanEval/95 (`check_dict_case`), where Python's `all()` short-circuits a generator expression, triggering `GeneratorExit` (a `BaseException` subclass used for control flow, not a real error). The tracer marks this as `[ERROR]`, and the model reasonably but incorrectly predicts FAIL. All three seeds produce identical errors, confirming this is a representation-inherent limitation, not training stochasticity. The error is actionable: distinguishing `BaseException` subclasses from `Exception` subclasses in the tracer would eliminate it.

Error interpretability decreases monotonically down the hierarchy. VOTA's 7 errors include the 2 GeneratorExit cases plus 5 from loop noise confusion. Full trace's approximately 217 errors are scattered across problem types with no dominant pattern, characteristic of information overload. No-trace's approximately 296 errors follow a single systematic pattern: majority-class bias (under-predicting FAIL).

The error containment hierarchy (exception$(2) \subset$ VOTA$(7) \subset$ full_trace(217)) provides a third evidence dimension (alongside accuracy and calibration) for the two-tier diagnostic hierarchy: the highest-specificity signal produces not only the fewest errors but the most interpretable and most actionable ones.

4.9 Out-of-Domain Generalization

To assess whether the exception signal generalizes beyond Python, we evaluate the Qwen-7B exception-only model on Java and JavaScript traces formatted with the same `[ERROR]` structure (Table 9).

OOD accuracy averages 96.8%, only 3.1pp below in-domain performance and 21.6pp above the Python no-trace baseline (75.1%). The modest degradation likely reflects novel exception types (Java's `NullPointerException`, JavaScript's `TypeError`) absent from the Python training set, rather than a failure of the general fault detection mechanism.

Language	Accuracy	n
Python (in-domain)	99.83%	1190
Java (OOD)	97.2%	$\sim$500
JavaScript (OOD)	96.3%	$\sim$500

Table 9: Cross-language generalization of exception-only SFT (Qwen-7B). OOD accuracy remains above 96%, with only a 3.1pp mean gap versus in-domain, suggesting the model learns language-agnostic fault detection rather than Python-specific patterns.

5 Discussion

VOTA as fault indicator isolation. Our results reposition VOTA from a generic trace compression method to a fault indicator isolation mechanism. VOTA's value lies not in making traces shorter, but in separating high-specificity fault indicators (exception events) from low-specificity behavioral patterns (loop summaries). The evidence is clear: exception-only ($\geq$ VOTA) and removing assertion-point data from VOTA does not degrade performance (99.41% in both cases). VOTA's practical contribution is automatically extracting the diagnostic moment from noisy traces.

Signal-limited, not capacity-limited. A recurring pattern across experiments is that post-transition performance depends on signal availability, not model capacity. Qwen-1.5B outperforms Qwen-7B at tail-512 (95.5% vs. 93.5%), while the reverse holds pre-transition (74.8% vs. 79.5%). Once the diagnostic signal enters the observation window, even small models achieve strong performance. This suggests that verdict-optimized trace design matters more than scaling model parameters for this task.

Training stability as practical bottleneck. DeepSeek exception-only shows seed-dependent performance (91–99% across seeds) compared to Qwen's perfect stability (99.83% $\pm$ 0.00). Notably, DeepSeek seeds with near-identical training loss (0.1225 vs. 0.122) produce accuracy gaps of 8.2pp, concentrated entirely in fail-class recall (1.0 vs. 0.705). Token-level loss masks sample-level classification ability, a general phenomenon in class-imbalanced settings. For deployment, we recommend multi-seed validation and task-specific (not loss-based) checkpoint selection.

Addressing the "feature engineering" concern. One might argue that exception extraction is merely feature engineering rather than a scientific contribution. The universality of our findings argues against this characterization: if exception dominance were an artifact of a particular model or dataset, we would not observe identical S-curve phase transitions across three model families at two parameter scales. The sharp phase transition at 384–512 tokens, with gradient ratios of 7–10:1 and consistent location across all four models, reflects a structural property of how diagnostic information distributes in execution traces, not an engineering trick.

Limitations. Our study evaluates a single task type (verdict prediction) on a single data source (HumanEval + MBPP). While our cross-model validation provides generalization evidence across architectures, evaluation on additional datasets and task types (e.g., fault localization, repair) is needed to assess broader applicability. The VOTA cross-model comparison is confounded by training-time trace truncation on DeepSeek (91.7% of samples truncated), which inflates the apparent model-capability gap. OOD language generalization results are preliminary and require dedicated cross-language benchmarks for definitive claims.

6 Conclusion

We present the first systematic study of which execution trace components drive verdict prediction. Across four code LLMs from three model families, exception events, structurally simple fault indicators, consistently achieve the highest accuracy, best calibration, and most interpretable failure modes. A causal decomposition reveals that information overload (74%) dominates gradient dilution (26%) in explaining why full traces underperform, and component ablation uncovers a strict two-tier diagnostic hierarchy. Token budget analysis reveals a universal sharp phase transition confirming that diagnostic information is concentrated in a localized moment occupying roughly 400–500 tokens at the trace tail. For verdict-oriented trace design, we recommend extracting the highest-specificity diagnostic signal and discarding everything else.

References

[1] A. Ni, M. Allamanis, A. Cohan, Y. Deng, K. Shi, C. Sutton, and P. Yin. DCSH: Diverse code solution hints for programming tasks. In *Findings of EMNLP*, 2024.

[2] S. Jiang, Z. Zheng, B. Zheng, et al. TracePile: A large-scale trace-augmented dataset for code language models. In *NeurIPS*, 2025.

[3] J. Wang, G. Chen, and H. Mei. TRACED: Execution-aware pre-training for source code. In *ICSE*, 2024.

[4] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa. A survey on software fault localization. *IEEE Trans. Software Eng.*, 42(8):707–740, 2016.

[5] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen. CodeT: Code generation with generated tests. In *ICLR*, 2023.

[6] A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. In *ICML*, 2024.